R1.04 - Architecture du Système d'Exploitation : Processus, Filiation, Cycle de Vie et Signaux POSIX

Thème principal : Fonctionnement interne des processus sous Unix/Linux, mécanismes de création (fork/exec), ordonnancement et communication par signaux POSIX

Synthèse exécutive : Ce cours pose les fondements de l'exécution des programmes sous les systèmes d'exploitation de type Unix/Linux. Il explique la transition critique d'un programme statique (stocké sur disque) vers un processus dynamique (instance en mémoire RAM). Pour gérer la concurrence sur des processeurs possédant un nombre limité de cœurs physiques, le noyau s'appuie sur le multitâche préemptif et un ordonnanceur (scheduler) gérant un modèle à trois états (Élu, Prêt, Bloqué). Le cours détaille la structure interne des processus (PCB), leur généalogie stricte issue de systemd (PID 1), le mécanisme biphase de création via fork() et exec(), ainsi que le système universel de contrôle par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Multitâche et PCB

Pourquoi cette notion existe-t-elle ? Dans un ordinateur, il y a beaucoup plus d'applications en cours de fonctionnement (navigateur, terminal, musique, services système) que de cœurs processeurs physiques. Sans un mécanisme d'arbitrage centralisé, une seule application bloquée ou trop gourmande gèlerait l'intégralité de la machine. Le système d'exploitation résout ce problème grâce à deux piliers : 1. L'isolation mémoire : Transformer un fichier exécutable passif sur disque en un processus autonome doté de son propre espace d'adressage virtuel en mémoire RAM (découpé en zones : Code, Data, Heap pour l'allocation dynamique, et Stack pour les appels de fonctions). 2. Le multitâche préemptif : L'ordonnanceur (scheduler) du noyau attribue une tranche de temps très courte (un quantum CPU) à un processus. Dès que le quantum expire ou si le processus a besoin d'effectuer une entrée/sortie (ex: lire un fichier sur disque), l'ordonnanceur lui retire le processeur et le donne à un autre processus. Le cycle de vie d'une tâche s'articule impérativement autour de 3 états : - Élu (Running) : Le processus est en train d'exécuter ses instructions sur un cœur processeur. - Bloqué (Waiting/Blocked) : Le processus a demandé une opération d'entrée/sortie ou attend un événement (frappe clavier, paquet réseau, lecture disque). Il est retiré du CPU par le noyau. - Prêt (Ready) : Le processus a toutes ses données en RAM et n'attend plus que l'attribution d'un quantum CPU par l'ordonnanceur. Point d'attention majeur pour les contrôles : Lorsque l'opération d'entrée/sortie d'un processus Bloqué se termine, le processeur est souvent déjà occupé par une autre tâche. Par conséquent, le noyau remplace l'état du processus de Bloqué à PRÊT. Le processus entre dans la file d'attente et attend que l'ordonnanceur l'élise à nouveau. Il est techniquement impossible d'effectuer une transition directe de Bloqué à Élu.

- Programme vs Processus : Un programme est un fichier binaire passif stocké sur disque (/usr/bin/ls). Un processus est l'instance active chargée en mémoire RAM consommant des ressources (CPU, mémoire, descripteurs).
- Multitâche Préemptif & Ordonnanceur : Le noyau découpe le temps processeur en quanta de temps (quelques ms) et distribue/retire autoritairement la main aux processus pour exécuter plusieurs tâches de façon simultanée.
- Process Control Block (PCB) : Structure de données du noyau contenant la carte mémoire, les identifiants (PID, PPID, UID), les registres processeur et les descripteurs de fichiers ouverts d'un processus.
- Cycle de Vie (3 États) : Élu (Running), Prêt (Ready), Bloqué (Waiting). Règle d'or : Un processus Bloqué repasse TOUJOURS par l'état Prêt une fois son événement survenu ; il ne repasse JAMAIS directement à l'état Élu.

Cas concret enseignant : *Analogie du livre de recettes de cuisine donnée en cours par l'enseignant : Le programme stocké sur disque dur est le livre de recettes rangé dans une bibliothèque (passif, statique, n'occupant aucun ingrédient ni énergie). Le processus est le cuisinier en train d'exécuter la recette dans la cuisine (dynamique, occupant de l'espace sur le plan de travail / mémoire RAM, utilisant du temps de cuisson / cycles CPU et manipulant des ustensiles / descripteurs de fichiers).*

2. Généalogie, Arborescence et Mécanismes de Création : fork() et exec()

Sous Unix/Linux, aucun processus ne peut naître de nulle part. Tous les processus forment une arborescence généalogique stricte. Au démarrage du système (boot), le noyau initialise l'espace utilisateur en lançant un tout premier processus : systemd (PID 1). Lorsqu'un utilisateur tape une commande dans son terminal (par exemple 'ls' ou 'vim'), la création de ce nouveau processus s'effectue en deux phases distinctes et indissociables : Phase 1 : fork() Le shell interactif (processus père) réalise un appel système fork(). Le noyau crée alors un nouveau processus (le fils) qui est une copie conforme du père (même mémoire, mêmes variables, mêmes descripteurs de fichiers). Le fils reçoit un nouvel identifiant (PID), et son PPID prend la valeur du PID du père. Phase 2 : exec() Immédiatement après le fork, le processus fils appelle une fonction de la famille exec() (ex: execve). Le noyau écrase le contexte mémoire du fils (qui était la copie du père) et charge à la place le fichier exécutable du programme cible (ex: /usr/bin/ls). Le PID du fils reste identique, mais son code et ses données sont désormais ceux du nouveau programme.

- PID et PPID : Process ID (identifiant unique) et Parent Process ID (identifiant du père).
- Processus Racine : systemd (ou init sur anciennes distributions), toujours associé au PID 1.
- Appel système fork() : Duplique à l'identique le processus appelant (création du fils).
- Appel système exec() : Remplace l'image mémoire du processus par le code binaire d'un nouveau programme.

Cas concret enseignant : *Exemple d'exécution dans un terminal Bash : Quand l'étudiant saisit la commande 'vim rapport.txt' dans son shell (PID 2048), Bash fait un fork() pour générer un fils (PID 2049, PPID 2048), puis ce fils fait un exec('/usr/bin/vim') pour devenir l'éditeur Vim.*

3. Communication Inter-Processus et Contrôle par Signaux POSIX

Les signaux POSIX représentent le mécanisme de communication IPC (Inter-Process Communication) le plus simple et le plus rapide sous Unix. Un signal agit comme une interruption logicielle : dès qu'un signal arrive, le processus interrompt momentanément le fil normal de ses instructions. Face à la réception d'un signal, un processus a trois options : 1. Action par défaut : Laisser le noyau appliquer le comportement par défaut (souvent la terminaison du processus). 2. Ignorer : Ignorer le signal (impossible pour certains signaux système). 3. Intercepter (Catch) : Exécuter un 'Signal Handler' personnalisé. Par exemple, un traitement de texte intercepte SIGTERM pour sauvegarder automatiquement le travail avant de se fermer. EXCEPTION CRITIQUE D'EXAMEN : L'autorité de l'administrateur et du noyau doit être préservée. C'est pourquoi deux signaux spécifiques ne peuvent JAMAIS être interceptés, masqués ou ignorés : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont traités directement par le noyau Linux sans passer par le code de l'application.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus.
- Signal Handler (Gestionnaire) : Fonction personnalisée écrite par le développeur pour intercepter (catch) un signal.
- SIGINT (Signal 2) : Envoyé par Ctrl+C, demande une interruption modérée.
- SIGTERM (Signal 15) : Envoyé par défaut par 'kill ', demande une extinction propre.
- SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel exécuté par le noyau, incapturable et inignorable.
- SIGSTOP (19) / SIGCONT (18) : Suspension par Ctrl+Z et reprise via 'fg' ou 'bg'.

Cas concret enseignant : *Analogie de l'expulsion du bâtiment donnée par le professeur : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte et vous demande d'évacuer (vous avez le temps de fermer vos fenêtres, éteindre les lumières et ranger vos affaires). SIGKILL (Signal 9) est l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous projette dehors instantanément sans vous laisser le temps d'attraper votre manteau.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Sans le tiret '-', l'utilitaire 'kill' n'interprète pas '9' comme un numéro de signal, mais comme un premier identifiant de processus (PID 9) auquel il essaie d'envoyer le signal par défaut (SIGTERM 15). Il tente ensuite d'envoyer le signal par défaut au PID 1234. Si vous n'avez pas les droits sur le PID 9, le système renvoie une erreur. Le tiret '-9' est indispensable pour indiquer à la commande qu'il s'agit du paramètre spécifiant le signal SIGKILL.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie est une tâche qui a terminé son exécution mais dont le père n'a pas encore lu le code de retour (via l'appel système wait). Le processus zombie n'occupe plus de mémoire RAM ni de CPU, mais son entrée persiste dans la table des processus (PCB). Il est STRICTEMENT IMPOSSIBLE de détruire un zombie avec 'kill -9 ' car le processus est DÉJÀ mort ! Pour le faire disparaître, il faut forcer son père à lire son statut, ou tuer le père (le fils zombie sera réadopté par systemd PID 1 qui le nettoiera immédiatement).

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 4521 Explicitez le rôle du symbole '&', détaillez le mécanisme de traitement du signal SIGTERM (15) et indiquez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Démarche de résolution :
Étape 1 : Le symbole '&' en fin de commande indique au shell d'exécuter la tâche 'sleep 600' en arrière-plan (background). Le shell affiche l'identifiant du job et le PID attribué (ex: 4521) puis rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' interroge la table des processus du noyau. Elle affiche une ligne confirmant que le processus avec PID 4521 exécutant '/bin/sleep 600' est présent en mémoire (état Prêt/Bloqué sur le timer de 600s), et affiche son PPID (qui correspond au PID du shell Bash). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. N'ayant aucun gestionnaire de signal personnalisé, le processus applique l'action par défaut du système : l'extinction propre et la libération de toutes les ressources.

Résultat : Le processus sleep 600 (PID 4521) est détruit proprement par le signal SIGTERM. La commande 'ps -ef | grep sleep' exécutée ultérieurement ne renvoie plus le processus 4521, confirmant son retrait définitif de la table des processus actifs du noyau.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le processus s'appelle systemd (ou init sur anciennes distributions) et son PID est impérativement 1.

Q : Quel signal POSIX est transmis au processus au premier plan lors du raccourci clavier 'Ctrl + C' ?

R : Le signal SIGINT (Signal numéro 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il pas être intercepté par un programme applicatif ?

R : Parce que SIGKILL est géré directement par le noyau Linux sans être transmis au code de l'application utilisateur, garantissant à l'administrateur le pouvoir absolu de stopper un processus.